GEO5017 - Assignment 2: Classification

Lars van Dijk (5571308), Niranjan Pradeep (6401643) and Thys Kok (5591406)

March 24, 2026

1 Introduction

Urban environments consist of diverse 3D objects such as buildings, lampposts, mailboxes, and vegetation. With the increasing availability of airborne LiDAR data, these datasets can be used for applications like urban modeling, autonomous driving, and environment monitoring. LiDAR point clouds provide accurate 3d measurements. Classification can be used to assign each point cloud a meaningful semantic label (e.g., building, tree, car). This report focuses on classifying urban objects using supervised learning. The goal is to design effective features and evaluate the performance of Support Vector Machines (SVM) and Random Forests (RF).

2 Methodology

500 point cloud files along with their corresponding labels were given as data to train and test the model. Their classes are building, fence, car, tree and pole. To be able to recognise the different classes, it is essential to define distinct features to help effectively differentiate between them.

Per point cloud, the position of all the points is given without any additional attributes. Therefore, the positions of a point cloud are used to construct Eigenvalues, which can be used to extract meaningful features about the overall point distribution of the item. It is important to note that the Eigenvalues were numbered based on their size according to: $\lambda_1 \geq \lambda_2 \geq \lambda_3$. Furthermore, a few simple additional features were also defined, such as the relative height. In total, eight features were defined, all of which are listed below:

- **Linearity:** Geometric feature that measures how strongly a point cloud is distributed along a single dominant direction, indicating a line like structure.

$$Linearity = \frac{\lambda_1 - \lambda_2}{\lambda_1}$$

- **Sphericity:** Geometric feature that measures how uniformly a point cloud is distributed in all directions.

$$Sphericity = \frac{\lambda_3}{\lambda_1}$$

- **Planarity:** Geometric feature showing how well a point cloud fits a flat surface.

$$Planarity = \frac{\lambda_2 - \lambda_3}{\lambda_1}$$

- **Verticality:** Measures how vertically oriented a surface or structure is by analyzing the direction of its normal vector relative to the vertical (z-axis).

$$Verticality = 1 - |n_z|$$

  where n_z is the z-component of the unit normal vector.

- **Hull area:** Area of the 2D convex hull enclosing the projected point cloud in the xy-plane.
- **Shape index:** Hull area divided by the hull perimeter.

- **Relative Height:** vertical extent of the point cloud, computed as the difference between its maximum and minimum z-values.

$$Relative\ Height = Z_{max} - Z_{min}$$

- **Curvature change:** Measures how fast the curvature changes.

$$CurvatureChange = \frac{\lambda_1}{\lambda_1 + \lambda_2 + \lambda_3}$$

Before selecting the best features, each feature was first normalised to ensure parity in contribution between different features. To identify the most effective features, a forward feature selection strategy was applied using the J criterion as an evaluation metric. Forward selection is used instead of selecting the top features individually because the J criterion evaluates the quality of a feature set, not isolated features. Features that perform well individually may be redundant when combined, as they can capture similar information. Forward selection accounts for these interactions by iteratively selecting features that maximize the J criterion for the current subset. This ensures that each selected feature contributes complementary information, leading to better overall class separability while avoiding unnecessary complexity.

The J criterion is defined as:

$$J = \frac{tr(S_b)}{tr(S_w)}$$

where S_b is **between-scatter matrix** and S_w is **within-scatter matrix**.

The selection procedure is defined as follows:

1. Compute the J criterion for each individual feature and select the feature with the highest score.
2. Iteratively evaluate all remaining features in combination with the selected subset.
3. At each step, select the feature that maximizes the J criterion when added to the current subset.
4. Repeat the process until four features are selected.

Using this procedure, the following feature subset was obtained see table 1

Table 1: Forward feature selection using the J criterion

Step	Selected Feature	J value
1	Relative Height (6)	2.39
2	+ Area (4)	1.64
3	+ Curvature Change (7)	1.35
4	+ Sphericity (1)	1.23

The results show that Relative Height (feature 6) dominates the J criterion with a value of 2.39, while the remaining features have relatively low individual scores. Notably, Area (feature 4) has a very low individual J value (0.15) and would not have been selected if features were ranked purely based on their individual performance.

However, once feature 6 is selected, adding feature 4 results in the highest J value among all candidate combinations (1.64). This demonstrates that feature 4 provides complementary information that is not captured by feature 6 alone. In other words, although feature 4 is weak in isolation, it becomes highly valuable in combination.

The other selected features provide complementary information. Relative height and area capture global geometric properties, while curvature change and sphericity describe local structural characteristics. Together, they improve class separability while limiting redundancy between features. The effectiveness of the selected feature subset is illustrated in Figure 1, based on the first two selected features.

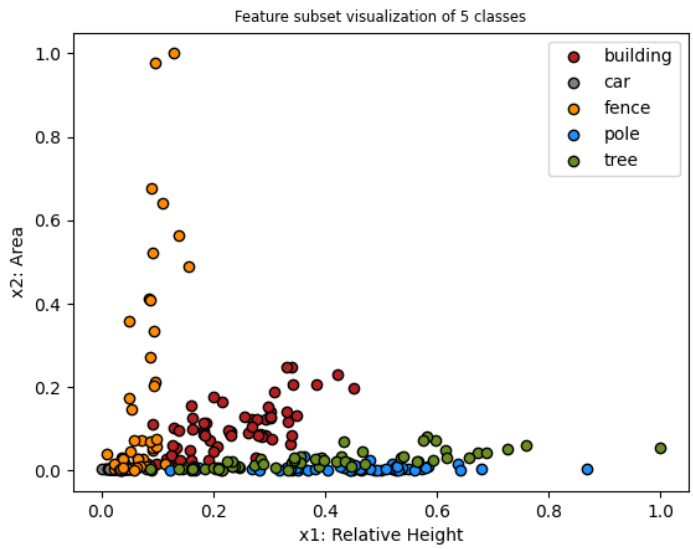


Figure 1: Feature subset visualization of classes for features relative height and area.

3 Experiments and evaluation

3.1 Hyperparameter Analysis

Both classifiers were evaluated using different combinations of hyperparameters to determine the optimal configuration based on their performance.

3.1.1 Support Vector Machine

For the SVM, different kernel functions (linear, polynomial, radial basis function, and sigmoid) were tested in combination with varying values of C, γ, and degree (for the polynomial kernel).

The hyperparameter experiments show clear differences in performance between kernel types and parameter settings. The linear kernel performs relatively poorly, with accuracy increasing from 0.40 to 0.74 as C increases, indicating that the data is not linearly separable and requires more flexible decision boundaries.

The polynomial kernel shows moderate performance, but is highly sensitive to the choice of γ. For most values of γ (0.01 and 0.1), performance drops significantly (around 0.22–0.25), indicating that the model fails to capture meaningful structure. Only the default $\gamma = \text{scale}$ produces reasonable results, with accuracy up to 0.74. Increasing the degree beyond 3 does not improve performance and often slightly degrades it, suggesting overfitting.

The RBF kernel consistently outperforms the other kernels. The best performance was achieved with $C = 100$ and $\gamma = 1$. However, this configuration represents a highly flexible model with minimal regularization, which increases the risk of overfitting.

A slightly lower value of $C = 10$ was selected for the final model, as it provides comparable performance while maintaining better regularization. This results in a more stable model with improved generalization, avoiding unnecessary model complexity.

Overall, the results show that the classification problem is non-linear and benefits from the flexibility of the RBF kernel. The optimal configuration provides a balance between model complexity and generalization.

3.1.2 Random Forest

The hyperparameter experiments for the Random Forest show that performance is relatively stable across different parameter settings.

Varying the number of trees ($n_estimators$) from 50 to 200 has only a minor effect on performance, indicating that even a relatively small ensemble is sufficient for this dataset. Increasing the number

of trees slightly improves stability, but does not lead to significant gains in accuracy.

The depth of the trees has a more noticeable effect. Limiting the depth to 5 or 10 results in similar or slightly better performance compared to deeper trees. Increasing the depth beyond this (e.g. 20 or no limit) does not improve accuracy and can slightly degrade performance, suggesting that deeper trees may lead to overfitting.

The choice of $max_features$ has only a small influence, log2 and sqrt perform similarly.

Based on these results, a configuration with $n_estimators = 100$, $max_depth = 5$, and $max_features =$ log2 was selected. This configuration provides strong performance while maintaining a relatively simple model, reducing the risk of overfitting.

3.2 Learning Curve Analysis

The learning curves show how model performance evolves as the size of the training set increases.

For both classifiers, performance improves as more training data is used, indicating that both models benefit from additional data (see figure 2). The Random Forest consistently achieves higher accuracy than the SVM across all training sizes.

The gap between training and test performance remains relatively small for both models, suggesting that both classifiers generalize well and do not suffer from severe overfitting. The Random Forest shows slightly higher training accuracy depending on the training set, indicating mild overfitting, but this does not significantly affect its test performance.

Overall, the learning curves indicate that both models are well-suited for the task, with Random Forest achieving more stable and consistently higher performance.

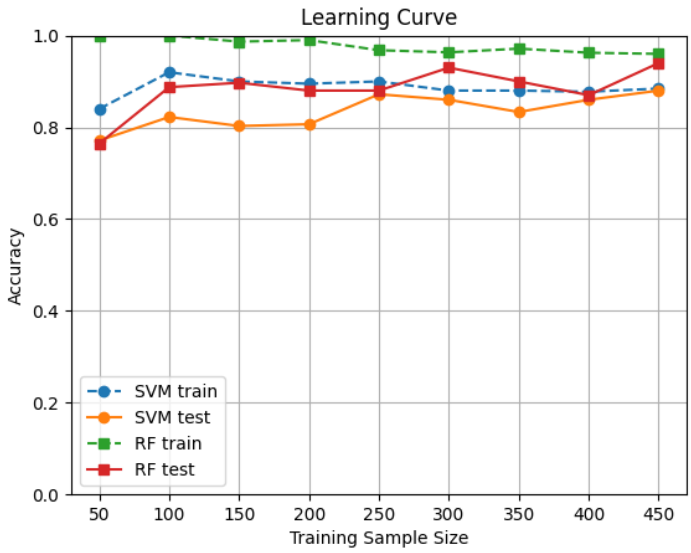


Figure 2: : Learning curve of classifiers

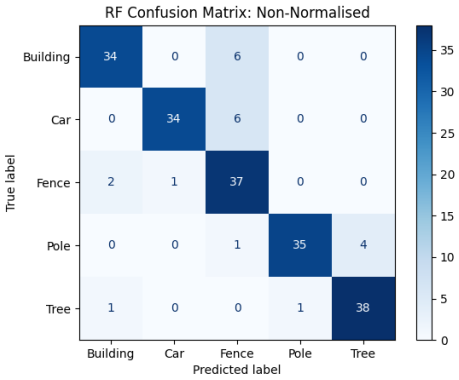
3.3 Error Analysis

To further evaluate model performance, confusion matrices were analysed for both classifiers.

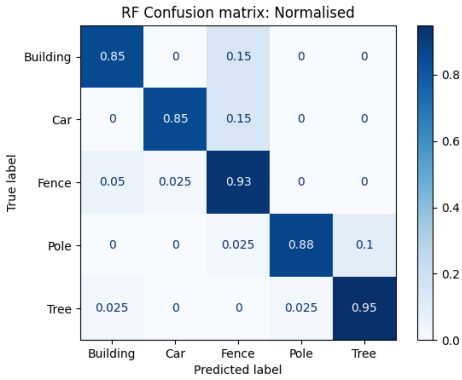
3.3.1 Random Forest

The Random Forest achieves strong performance across most classes, with particularly high accuracy for tree, fence, and pole classes. The majority of errors occur between building, car, and fence classes (see figure 3a, 3b).

This confusion can be explained by similarities in geometric properties. For example, fences and buildings both exhibit planar characteristics, while cars and fences can have similar spatial extents. Additionally, some confusion occurs between poles and trees, likely due to similar vertical structures.



(a) Non-normalised Confusion Matrix of the Random Forest



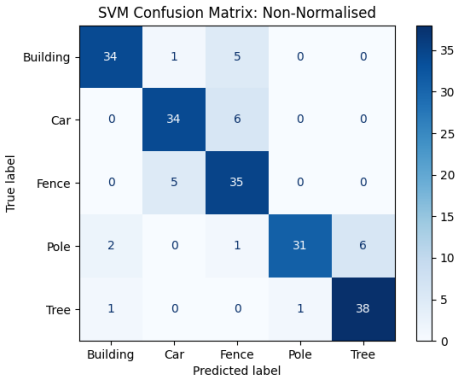
(b) Normalized Confusion Matrix of the Random Forest

Figure 3: Confusion matrices, Random Forest

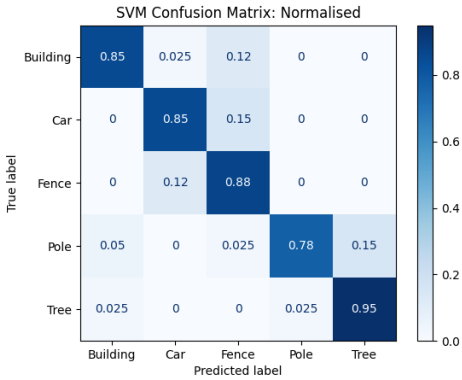
3.3.2 Support Vector Machine

The SVM shows a similar pattern of errors but with slightly lower performance, particularly for the pole and fence classes. The confusion between pole and tree is more pronounced, indicating that the SVM struggles more with separating classes that overlap in feature space (see figure 4a, 4b).

Overall, the SVM is more sensitive to overlapping features, while the Random Forest handles these ambiguities more effectively due to its ensemble structure.



(a) Non-normalised Confusion Matrix of the Support Vector Machine



(b) Normalised Confusion Matrix of the Support Vector Machine

Figure 4: Confusion Matrices, Support vector machine

3.4 Comparison of Classifiers

The macro-averaged F1-score was used for comparison, as it balances precision and recall while giving equal weight to each class, thereby highlighting differences in class-wise performance rather than only overall accuracy. The Random Forest outperforms the SVM in F1-score, achieving 0.89 compared to 0.86 for the SVM (see table 2).

Additionally, the Random Forest shows more stable performance across different hyperparameter settings and training datasets, while the SVM requires careful tuning to achieve optimal results.

Overall, Random Forest provides better performance and robustness for this classification task.

Table 2: Selected final models and performance

Model	Parameters	Accuracy	F1-score
SVM	RBF, $C = 10$, $\gamma = \text{scale}$	0.86	0.861
RF	$n = 100$, depth=5, $max_features = \log 2$	0.89	0.892

4 Conclusion

It was observed that the results can vary significantly depending on the train-test split when no fixed `random_state` is used, with accuracy ranging approximately between 0.75 and 0.95. This variability indicates that the performance of both classifiers is sensitive to the specific data split, which is expected given the relatively small dataset size. Despite this variation, the Random Forest consistently outperforms the SVM across different runs. This suggests that the Random Forest is more robust to variations in the training data. The observed variability also indicates that increasing the size of the training dataset would likely improve the stability and reliability of the results. With more data, the model can better capture the underlying data distribution, reducing sensitivity to individual samples.

Finally, the feature selection results show that, apart from Relative Height, most features have relatively low individual J values, indicating limited discriminative power on their own. Even after combining features, the overall J value remains moderate and decreases as more features are added, suggesting that the selected feature set does not achieve strong class separability. This indicates that the current features only partially capture the differences between object classes. Therefore, further improvement could be achieved by designing more discriminative features that better represent geometric and structural differences between classes.

5 Code

Link to GitHub project repository.

6 Division of Work

Table 3: Division of the Tasks

Lars van Dijk (40%)	**Niranjan Pradeep (30%)**	**Thys Kok (30%)**
Forward selection implementation	Definition of Initial Features	Definition of initial features
Hyperparameter tuning	Plotter & Confusion matrix functions	Feature normalization
Comparison of classifiers using F1-score	Random forest implementation	Feature selection initial implementation
General report writing	General report writing	General report writing